

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

Полиморфизм

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы	3
2 Задание	3
3 Описание работы программы.....	4
4 Исходный код программы.....	7
5 Результаты работы программы	18
6 Вывод	19

1 Цель работы

Изучение механизма полиморфизма в объектно-ориентированном программировании, с акцентом на виртуальные методы и свойства, их переопределение, запрет на переопределение, а также альтернативные способы изменения функциональности базовых классов, включая механизм сокрытия.

2 Задание

Изучить переопределение методов и свойств. Реализовать виртуальные методы и свойства в базовом классе с использованием ключевого слова «virtual». В производных классах переопределить эти методы и свойства.

Изучить модификатор «sealed» для ограничения дальнейшего переопределения методов или свойств в производных классах. Исследовать поведение программы при использовании «sealed» и описать его.

Изучить альтернативу переопределению — механизм сокрытия. с переопределением («override») в контексте поведения программы при преобразовании типов.

3 Описание работы программы

В классе Car реализован виртуальный метод GetCarInfo, который можно переопределить в производных классах. В классе Info, который наследуется от Car, метод GetCarInfo переопределён с использованием ключевого слова override для отображения более подробной информации об автомобиле. При вызове метода через полиморфизм из формы Windows Forms будет определяться тип объекта, и вызовется соответствующая версия метода. Если пользователь выбирает машину с общими характеристиками, будет вызван метод из класса Car, а если пользователь выберет детализированную информацию, метод из класса Info будет выполнен.

```
Ссылка 3
public virtual string GetCarInfo(bool showDetailed = false)
{
    // Основная информация о машине
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    // Если showDetailed == true, добавляется подробная информация

    return info;
}
```

Рисунок 1 – Метод virtual для перегрузки метода

Также был исследован модификатор sealed, который запрещает дальнейшее наследование и переопределение методов. В классе Info модификатор sealed ограничивает возможность наследования и изменения его функциональности в других производных классах, что позволяет защитить определённую логику от случайного или ненамеренного изменения. При попытке наследования от класса с модификатором sealed программа выдаст ошибку компиляции, демонстрируя его поведение.

```
namespace CarManagementApp
{
    Ссылка 3
    public sealed class Info : Car
    {
        // Конструктор для класса Info, который вызывает конструктор базового класса
        Ссылка 1
        public Info(string name, int horsePower, int fuelConsumption, int fuelTankCapacity)
            : base(name, horsePower, fuelConsumption, fuelTankCapacity)
        {
        }

        // Переопределяем метод получения информации о машине
        Ссылка 3
        public override string GetCarInfo(bool showDetailed = false)
        {
            // ...
        }
    }
}
```

Рисунок 2 – Метод sealed для ограничения метода

Кроме того, был изучен альтернативный механизм изменения функциональности через сокрытие. Сокрытие реализуется с использованием ключевого слова `new`, которое позволяет скрыть метод из базового класса в производном. Это не влияет на поведение при вызове метода через переменную базового типа, в то время как `override` полностью изменяет функциональность метода при использовании полиморфизма.

```
public override string GetCarInfo(bool showDetailed = false)
{
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(" ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}")}";

    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
```

Рисунок 3 – Метод override для перегрузки

Для работоспособности приложения были переписаны стандартные методы `on_click()`. Для метода `AddCar` была добавлена диагностика на проверку типа объекта. В свою очередь `ShowInfoCar` переписана для проверки активного чекбокса и проверки в момент добавления авто.

```

private void buttonShowInfo_Click(object sender, EventArgs e)
{
    try
    {
        // Проверяем, был ли выбран автомобиль
        string selectedCar = listBoxCars.SelectedItem?.ToString();
        if (selectedCar != null)
        {
            Car car = carPark.GetCar(selectedCar);

            if (car == null)
            {
                MessageBox.Show("Ошибка: Автомобиль не найден.");
                return;
            }

            // Добавляем диагностику типа объекта
            if (car is Info infoCar)
            {
                // Если это объект типа Info, используем метод Info
                bool showDetails = checkBoxShowDetails.Checked;
                string info = infoCar.GetCarInfo(showDetails);
                MessageBox.Show(info);
            }
            else
            {
                // В противном случае используем метод базового класса
                bool showDetails = checkBoxShowDetails.Checked;
                string info = car.GetCarInfo(showDetails);
                MessageBox.Show(info);
            }
        }
        else
        {
            MessageBox.Show("Выберите автомобиль для отображения информации.");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при отображении информации: {ex.Message}");
    }
}

```

Рисунок 4 – Обновленный метод получения

```

Ссылка: 1
private void buttonAddCar_Click(object sender, EventArgs e)
{
    try
    {
        string name = textBoxCarName.Text;
        int horsepower = int.Parse(textBoxHorsePower.Text);
        int fuelConsumption = int.Parse(textBoxFuelConsumption.Text);
        int fuelTankCapacity = int.Parse(textBoxFuelTankCapacity.Text);

        // Создаем объект типа Info вместо базового Car
        var car = new Info(name, horsepower, fuelConsumption, fuelTankCapacity);

        carPark.AddCar(car);

        listBoxCars.Items.Add(name);
        listBoxActions.Items.Add($"Добавлен автомобиль: {name}");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при добавлении автомобиля: {ex.Message}");
    }
}

```

Рисунок 5 – Обновленный метод добавления

На рисунке 6 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area contains several controls:

- On the left, there are four text labels with corresponding input fields: "Введите имя авто:", "Введите мощность авто:", "Введите расход авто:", and "Введите бак авто:". Below these are two buttons: "Создать авто" and "Удалить авто".
- In the center, there is a large empty rectangular box.
- On the right, there are five buttons stacked vertically: "Заправка", "Передвижение", "Запустить", "Выключить", and "Заменить".
- Below the "Создать авто" button, there is a checkbox labeled "Подробно" and a button labeled "Показать".
- At the top right, there is a checkbox labeled "Высокая скорость".

Рисунок 6 – Оформление окна Windows Forms

4 Исходный код программы

Файл «Info.cs»:

```
using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {
        // Конструктор для класса Info, который вызывает конструктор базового класса
        public Info(string name, int horsePower, int fuelConsumption, int fuelTankCapacity)
            : base(name, horsePower, fuelConsumption, fuelTankCapacity)
        {
        }

        // Переопределяем метод получения информации о машине
        public override string GetCarInfo(bool showDetailed = false)
        {
            string info = $"Название: {Name}\n" +
                $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
                $"Пробег: {Mileage} км\n" +
                $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
                $"Расход топлива: {FuelConsumption} л/100 км\n" +
                $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
                $"Колеса: {string.Join(" ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

            if (showDetailed)
            {
                info += "\nОставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
                for (int i = 0; i < Wheels.Length; i++)
                {
                    info += "\nКолесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
                }
            }

            return info;
        }
    }
}
```

Файл «Car.cs»:

```
using System;
using System.Linq;

namespace CarManagementApp
{
    // Класс Car представляет автомобиль с его компонентами (двигатель, колеса) и управлением ими
    public class Car
    {
        // Свойство Name хранит название автомобиля с обработкой лишних пробелов
        private string _name;
        public string Name
        {
            get => _name;
            set => _name = value?.Trim() ?? throw new ArgumentException("Название не может быть пустым.");
        }

        // Свойство Engine хранит объект двигателя автомобиля
        public Engine Engine { get; set; }

        // Свойство Wheels хранит массив колес автомобиля (массив из 4 колес)
        public Wheel[] Wheels { get; set; }

        // Свойство Fuel хранит текущее количество топлива в автомобиле с проверкой на отрицательные значения
        private int _fuel;
        public int Fuel
        {
            get => _fuel;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Количество топлива не может быть отрицательным.");
                _fuel = value;
            }
        }

        // Свойство FuelTankCapacity хранит максимальный объем топливного бака
        public int FuelTankCapacity { get; private set; }

        // Свойство Mileage хранит пробег автомобиля в километрах
        public int Mileage { get; private set; } = 0;

        // Свойство FuelConsumption хранит расход топлива на 100 км
        public int FuelConsumption { get; private set; }

        // Статическое свойство для общего количества колес
        public static int WheelCount { get; } = 4;

        // Свойство Symbol проверяет регистр символа и преобразует строчные буквы в прописные
        private char _symbol;
        public char Symbol
        {
            get => _symbol;
            set => _symbol = char.IsLower(value) ? char.ToUpper(value) : value;
        }

        // Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность
        private DateTime _lastServiceDate;
        public DateTime LastServiceDate
        {
            get => _lastServiceDate;
```

```

set
{
    if (value < DateTime.Now)
        throw new ArgumentException("Дата не может быть в прошлом.");
    _lastServiceDate = value;
}
}

// Конструктор класса Car
// Инициализирует автомобиль с заданными параметрами: имя, мощность двигателя, расход топлива и объем бака
public Car(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
{
    Name = name;

    Engine = new Engine { HorsePower = horsepower };
    Wheels = new Wheel[4]; // Инициализация массива колес

    // Инициализация колес и подписка на событие поломки каждого колеса
    for (int i = 0; i < WheelCount; i++)
    {
        Wheels[i] = new Wheel();
        Wheels[i].ComponentBroken += OnComponentBroken; // Подписка на событие поломки колеса
    }

    Engine.ComponentBroken += OnComponentBroken; // Подписка на событие поломки двигателя

    FuelConsumption = fuelConsumption;
    FuelTankCapacity = fuelTankCapacity;
    Fuel = FuelTankCapacity; // Заполнение бака до максимума
}

// Свойство IsEngineOn проверяет, работает ли двигатель
// Возвращает true, если двигатель включен, иначе false
public bool IsEngineOn => Engine.IsRunning;

// Метод StartEngine запускает двигатель
public void StartEngine() => Engine.Start();

// Метод StopEngine выключает двигатель
public void StopEngine() => Engine.Stop();

// Обработчик события поломки компонента (колеса или двигателя)
// При поломке компонента выводится сообщение о поломке в консоль
private void OnComponentBroken(object sender, EventArgs e)
{
    if (sender is Engine)
    {
        // Сообщение о поломке двигателя
        Console.WriteLine($"{Name}: Двигатель сломан!");
    }
    else if (sender is Wheel wheel)
    {
        // Сообщение о поломке колеса
        Console.WriteLine($"{Name}: Колесо {Array.IndexOf(Wheels, wheel) + 1} сломано!");
    }
}

// Перегрузка метода Move: стандартное движение (обычная скорость)
// Этот метод вызывает второй метод Move с параметром isFast = false (стандартная скорость)

```

```

public void Move(int distance)
{
    Move(distance, false); // Перемещение с обычной скоростью
}

// Перегрузка метода Move: движение с дополнительными параметрами (например, высокая скорость)
// Если параметр isFast равен true, расход топлива снижается на 20% (ускоренный режим)
public void Move(int distance, bool isFast = false)
{
    // Проверка, включен ли двигатель, если выключен - нельзя двигаться
    if (!IsEngineOn)
    {
        throw new InvalidOperationException("Двигатель выключен! Невозможно передвигаться.");
    }

    // Проверка, не сломан ли двигатель
    if (Engine.Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, машина не может двигаться.");

    // Проверка, достаточно ли топлива для движения
    if (Fuel < (distance * FuelConsumption) / 100)
        throw new InvalidOperationException("Недостаточно топлива!");

    // Проверка, не сломано ли одно из колес
    if (Wheels.Any(w => w.Condition == "Сломано"))
        throw new InvalidOperationException("Одно из колес сломано!");

    // Увеличиваем пробег
    Mileage += distance;

    // Расчет расхода топлива в зависимости от скорости
    if (isFast)
    {
        // Ускоренная скорость: расход топлива снижается на 20%
        Fuel -= (int)((distance * FuelConsumption) / 100 * 0.8); // Приведение типа к int
    }
    else
    {
        // Обычная скорость: расход топлива без изменений
        Fuel -= (distance * FuelConsumption) / 100;
    }

    // Поломка колеса каждые 200 км
    if (Mileage % 200 == 0)
    {
        Random random = new Random();
        int brokenWheelIndex = random.Next(Wheels.Length);
        Wheels[brokenWheelIndex].ChangeCondition("Сломано"); // Случайная поломка одного из колес
    }

    // Поломка двигателя каждые 1000 км
    if (Mileage % 1000 == 0)
    {
        Engine.ChangeCondition("Сломано"); // Поломка двигателя после 1000 км пробега
    }
}

// Метод Refuel заправляет автомобиль на указанное количество литров
// Проверяет, не превышает ли заправка емкость бака
public void Refuel(int amount)
{

```

```

        if (amount <= 0)
            throw new ArgumentException("Заправляемое количество должно быть положительным!");

        Fuel += amount;
        if (Fuel > FuelTankCapacity)
            Fuel = FuelTankCapacity; // Если заправлено больше, чем вмещает бак, устанавливаем максимальную емкость
    }

    // Метод ReplaceBrokenWheel заменяет сломанное колесо на новое
    public void ReplaceBrokenWheel()
    {
        for (int i = 0; i < Wheels.Length; i++)
        {
            if (Wheels[i].Condition == "Сломано")
            {
                Wheels[i].ChangeCondition("Новое"); // Заменить сломанное колесо
                return;
            }
        }
        throw new InvalidOperationException("Нет сломанных колес для замены."); // Если нет сломанных колес
    }

    // Метод для получения информации о машине
    // Если параметр showDetailed = true, выводится подробная информация
    public virtual string GetCarInfo(bool showDetailed = false)
    {
        // Основная информация о машине
        string info = $"Название: {Name}\n" +
            $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
            $"Пробег: {Mileage} км\n" +
            $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
            $"Расход топлива: {FuelConsumption} л/100 км\n" +
            $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
            $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

        // Если showDetailed == true, добавляется подробная информация

        return info;
    }
}

```

5 Результаты работы программы с примерами

На рисунке 7 представлен результат работы программы с методом класса вывода информации.

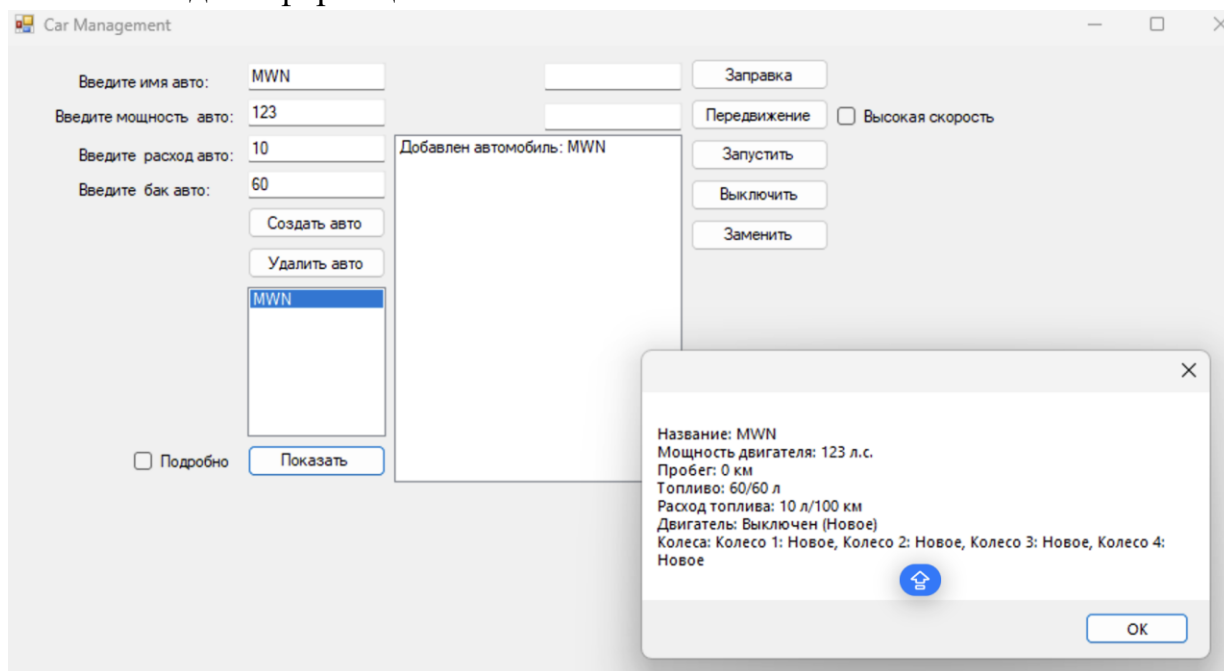


Рисунок 7 – Результат работы программы

На рисунке 8 представлен результат работы программы с подклассом для юридического лица.

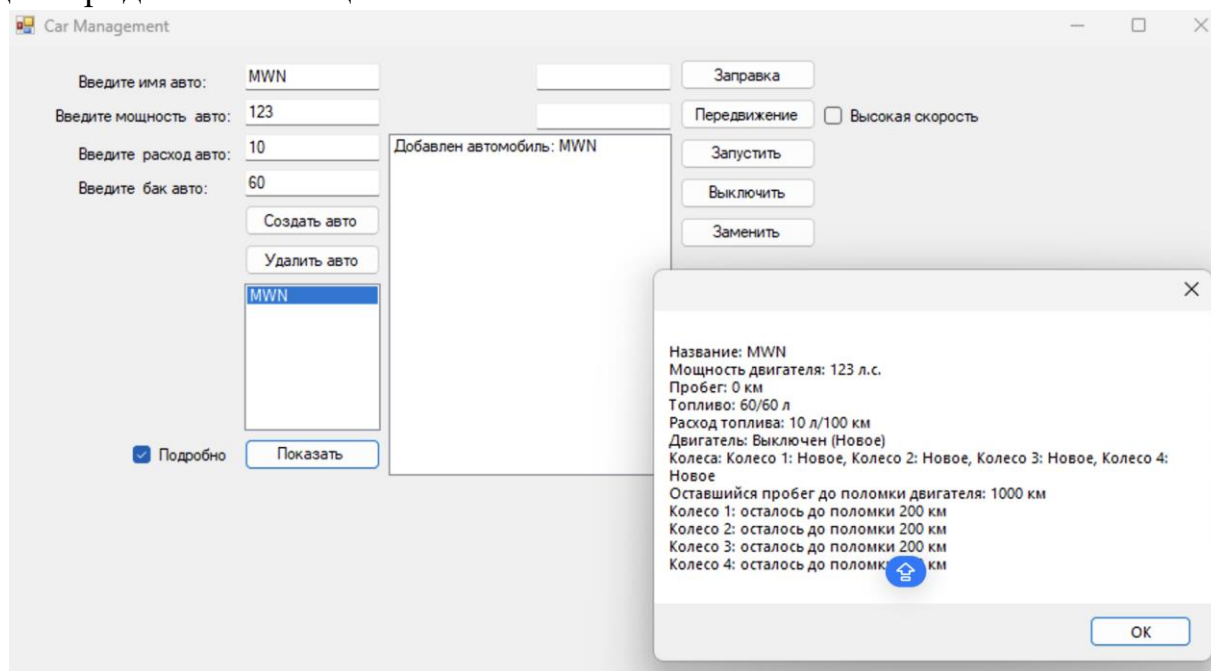


Рисунок 8 – Результат работы программы

6 Вывод

В ходе выполнения работы была разработана программа, которая включает несколько классов с различной функциональностью и конструкторами. Реализованы виртуальные методы и свойства в базовом классе с использованием ключевого слова «virtual», а также их переопределение в производных классах с помощью «override». Это продемонстрировало возможности изменения функциональности базового класса и использования полиморфизма.

Изучен модификатор «sealed» для ограничения дальнейшего переопределения методов и свойств в производных классах. Исследовано поведение программы при использовании данного модификатора, что позволило понять его значение в проектировании классов.

Исследован механизм как альтернатива переопределению («override»). Проведен сравнительный анализ поведения программы при сокрытии и переопределении методов, в том числе при преобразовании типов, что позволило выявить ключевые отличия этих механизмов.